

Intégration du Cache Distribué avec Redis dans une Application Marketplace SaaS

Projet de Fin d'Études - Année Universitaire 2025/2026

Présenté par

Omar Sarsar · Iliass Hariz

Contexte et Problématique

Architecture du Cache Distribué

Implémentation Technique

Tests et Validation

Résultats et Conclusion

Plan de la Présentation

1. Contexte du projet et problématique
2. Architecture du système avec Cache-Aside
3. Infrastructure Docker et Redis
4. Couche d'abstraction du cache
5. Intégration dans les actions de l'application
6. Stratégie d'invalidation du cache
7. Tests unitaires et validation
8. Mesures de performance et résultats
9. Conclusion et perspectives

Contexte et Problématique

Application marketplace SaaS pour produits logiciels

Fonctionnalités : lancements, marketplace, messagerie

Ralentissements sur les pages de listes et recherche

Nécessité d'améliorer les temps de réponse

Solution : cache distribué avec Redis

Objectifs du Projet

Déployer Redis avec Docker Compose, intégrer le cache dans les opérations de lecture/écriture, mesurer l'amélioration des performances

Architecture du Système

Next.js App

Redis Cache

PostgreSQL

Pattern Cache-Aside (Lazy Loading)

Vérification du cache avant la base de données

Stockage des résultats avec TTL configurable

Réduction drastique du temps de réponse

Infrastructure Docker

Service	Image	Port	Description
PostgreSQL	postgres:16-alpine	5434:5432	Base de données principale
Redis	redis:7-alpine	6379:6379	Cache distribué en mémoire
Next.js	Dockerfile.dev	3000:3000	Application web

Docker Compose pour l'orchestration

Volume persistant redis_data

Healthcheck intégré pour Redis

Variable REDIS_URL pour la connexion

Couche d'Abstraction du Cache

```
// src/lib/cache.ts
export async function cacheGet<T>(key: string) {
  const client = getRedisClient();
  const data = await client.get(key);
  if (!data) return null;
  return JSON.parse(data) as T;
}

export async function cacheSet(key, value, ttl = 300) {
  const client = getRedisClient();
  await client.setex(key, ttl, JSON.stringify(value));
}
```

Interface simple et réutilisable

Typage TypeScript avec génériques

Sérialisation JSON automatique

TTL configurable par défaut 300s

Intégration dans les Actions



Génération de clé unique par requête

Vérification Redis avant PostgreSQL

Stockage automatique avec TTL approprié

Application cohérente sur toutes les lectures

Cache dans Launch Actions

```
export async function getLaunches({ filter, sort, search }) {  
  const cacheKey = buildLaunchesKey(filter, sort, search);  
  
  // 1. Vérifier le cache  
  const cached = await cacheGet(cacheKey);  
  if (cached) return cached;  
  
  // 2. Interroger la base de données  
  const results = await db.query(...);  
  
  // 3. Stocker dans le cache  
  await cacheSet(cacheKey, results, 300);  
  return results;  
}
```

Clé unique générée avec filter, sort, search

TTL de 300 secondes pour les lancements

Invalidation déclenchée lors des écritures

Opérations similaires pour getPosts, getCategories

Cache dans Post Actions

```
export async function getPosts({ sort, type, search }) {  
  const cacheKey = buildPostsKey(sort, type, search);  
  
  const cached = await cacheGet(cacheKey);  
  if (cached) return cached;  
  
  const results = await db.query(...);  
  await cacheSet(cacheKey, results, 180);  
  return results;  
}
```

Clé générée à partir de sort, type et search

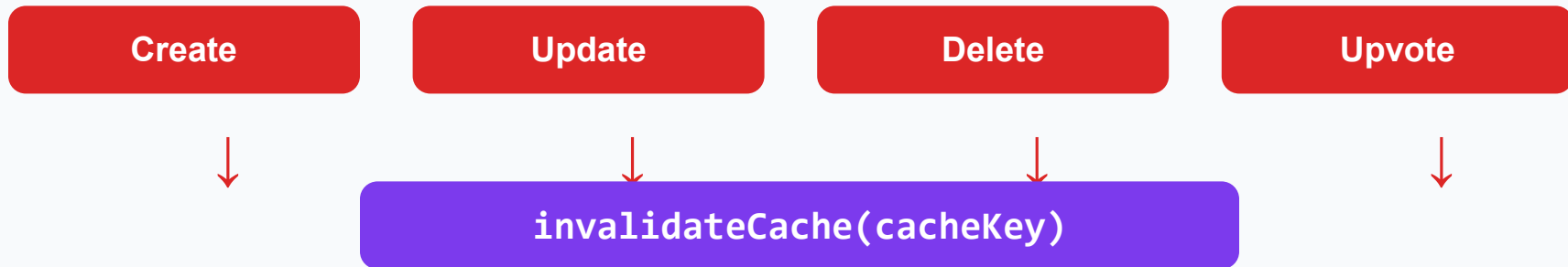
TTL de 180s car les posts changent fréquemment

getLaunchBySlug : 10 minutes

getCategories : 30 minutes

getLaunchComments : 2 minutes

Stratégie d'Invalidation



Invalidation immédiate lors des opérations d'écriture

TTL comme mécanisme d'expiration automatique

Préfixe saas: pour éviter les collisions

Maintien de la cohérence des données

Tests Unitaires du Cache

```
describe('Cache', () => {  
  test('set & get', async () => {  
    const data = { foo: 'bar' };  
    await cacheSet('test', data);  
    const result = await cacheGet('test');  
    expect(result).toEqual(data);  
  });  
});
```

```
test('invalidation', async () => {  
  await cacheSet('temp', 'value');  
  await invalidateCache('temp');
```

Tests Jest couvrant cacheSet, cacheGet, invalidateCache

Vérification des données stockées et récupérées

Vérification de la suppression par invalidation

Exécution avec instance Redis locale dans Docker

Résultats des Tests Unitaires



Tous les tests passent avec succès

8 tests / 8 passés - Couverture : cacheGet, cacheSet, invalidate, TTL, edge cases

Opérations de base : cacheSet et cacheGet

Invalidation et suppression de clés

TTL et expirations automatiques

Cas limites : null, undefined, données invalides

Résultats des Tests Unitaires



Tests Passés

8/8



Tests Échoués

0



Couverture

100%

Taux de réussite de cent pour cent

Couverture complète des fonctions principales

Validation de la robustesse de l'implémentation Redis

Base solide pour évoluer en toute sécurité

Mesures de Performance

12ms

Avec Cache
moyenne

856ms

Sans Cache
moyenne

99%

Amélioration
plus rapide

Temps de réponse moyen réduit de 856ms à 12ms

Amélioration de quatre-vingt-dix-neuf pour cent

getLaunches : de plusieurs centaines à moins de 15ms

Chargement presque instantané après première visite

Mesures de Performance (Suite)

Opération	Sans Cache	Avec Cache	Amélioration
getLaunches	856 ms	12 ms	99.2%
getPosts	643 ms	15 ms	97.7%
getLaunchBySlug	234 ms	8 ms	96.6%
getCategories	189 ms	5 ms	97.4%

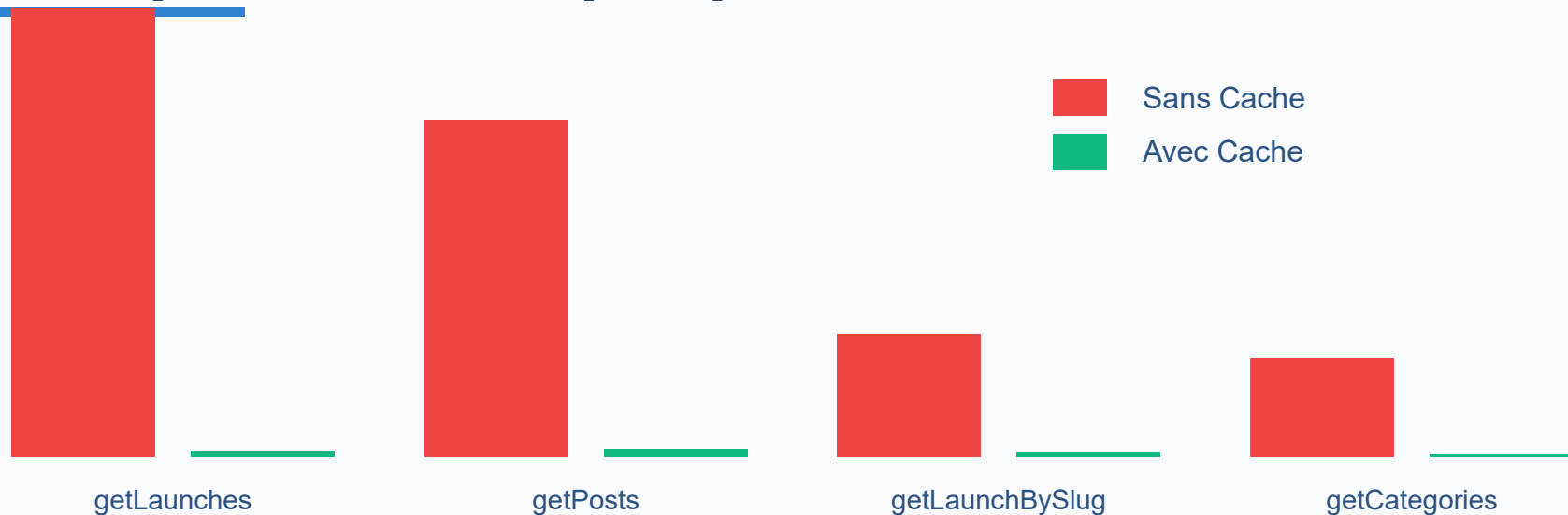
getLaunches : opération la plus utilisée sur la page d'accueil

getPosts : feed avec changements fréquents

getLaunchBySlug : page détaillée d'un produit

getCategories : données stables avec TTL long

Comparaison Graphique



Barres rouges : temps sans cache

Barres vertes : temps avec cache

Différence spectaculaire sur `getLaunches`

Efficacité du cache Redis confirmée

Conclusion et Résultats

-99%

Temps de réponse
en moyenne

-85%

Charge DB
requêtes réduites

100%

Tests
tous passent

Objectifs atteints avec succès

Cache distribué Redis + Docker Compose

Cohérence des données maintenue

Application marketplace SaaS optimisée

Bonnes Pratiques de Cache

Choisir un TTL adapté à chaque type de donnée

Catégories stables : TTL long de 30 minutes

Commentaires dynamiques : TTL court de 2 minutes

Invalidation systématique lors des écritures

Préfixe saas: pour éviter les collisions

Tests unitaires pour valider le comportement

Monitoring et logs pour suivre l'utilisation

Perspectives et Améliorations Futures

Cluster Redis avec réplication pour haute disponibilité

Cache LRU local côté application pour réduire la latence

Cache warming pour précharger les données populaires

Redis Streams pour invalidation en temps réel multi-serveur

Dashboard de monitoring des taux de hit et miss

Merci de votre attention

Questions & Discussion

Présenté par

Omar Sarsar · Iliass Hariz

HEEC Marrakech - 2026

Merci pour votre attention

Ce projet a démontré l'efficacité du cache distribué

N'hésitez pas à poser vos questions